DHBW Stuttgart

Karsten Keßler

Datenbanksysteme
11. Teil – Datenintegrität

Kurs Informatik 24ITA

Übersicht der Gesamtvorlesung

Thema	Inhalte (Auszug)
1 Einführung	Einführung, Begriffe, Eigenschaften Funktionen, Datenintegration
2 Datenbankmodelle	Architektur und SPARC, Verschiedene Datenbankmodelle, Entwicklung
3 ERM	Entity-Relationship-Modell, konzeptueller Entwurf, Entwurfsprozess
4 Relationales Modell	Relationales Datenbankmodell im Detail
5 Relationale Algebra	Operatoren, Anwendung, Nützlichkeit
6 Normalformen	Normalisierung von Datenbanken, funktionale Abhängigkeiten, Synthesealgorithmus
7 Vektordatenbanken	Einstieg in Vektordatenbanken
8 SQL DDL	Datendefinition mit SQL
9 SQL DML	Datenmanipulation mit SQL
10 Transaktionen	ACID, Grundlegende Schwierigkeiten bei Mehrbenutzersystemen, gängige Lösungen
11 Integrität	Integrität im relationalen Modell wiederholt, Umsetzung (in SQL)
12 Sichten	Sichten (in SQL), Vor- und Nachteile, Umgang mit Sichten
13 Indizes	Interne Strukturen, Nützlichkeit für Anfragen
14 Optimierung	Optimierung von Datenbanken
15 SQL Server	Administration des SQL Server
16 Handout	Handout: Datenbanksysteme – Zusammenfassung

Integritätsbedingungen

- **Entitätsintegrität**: In einer Relation gibt es keine zwei Tupel mit gleichem Schlüsselwert, „lokale Integrität“
  Bsp.: Zwei Kunden dürfen nicht dieselbe Kundennummer haben
- Schlüssel ist ein (häufiger) Spezialfall von lokaler Integrität.

- Man definiert damit ein erweitertes Relationenschema: $\mathbf{R_E = (R , I)}$

  Für jede Integritätsbedingung b ∈ I muss gelten:
  b(r) = true für r = Relation über R.

R_E = Erweitertes Relationenschema
R = Relationenschema
I = Integritätsbedingung

- Alle **Beziehungen** werden über **Werte** hergestellt (Wesensmerkmal für relationale Datenbanken):
  Fremdschlüssel: Attributmenge, die sich auf die Primärschlüsselattribute einer anderen Relation bezieht.

- **Referenzielle Integrität**: Wertekombination des Fremdschlüssels muss immer Schlüssel der Vaterrelation sein und die entsprechenden Werte müssen dort auch enthalten sein.
  Bsp.: Jede Bestellung muss auf einen existierenden Kunden beziehen.
- Beispiele für **Globalen Integritätsbedingung**

  Foreign Key Einschränkungen: Referenzierte Relation, auf die sich der Fremdschlüssel bezieht (z.B. jede Bestellung muss einem Kunden zugeordnet sein)

  Unique Einschränkungen: Bestimmte Werte in einer Tabelle müssen eindeutig sein (z.B. zwei Kunden können nicht denselben Benutzernamen haben, Email-Adresse darf pro Nutzer nur einmal vorkommen).

  Check Einschränkungen: Werte in einer Spalte müssen bestimmte Bedingungen entsprechen (z.B. nur positive Werte: preis > 0)

  Trigger Einschränkungen: Aktionen werden automatisch ausgeführt bei bestimmten Ereignissen (z.B. nach einem Update)

Integritätsbedingungen

Verstoß gegen ref. Integrität: Kunde 999 existiert nicht.

KundenID	Name	E-Mail
101	Müller	mueller@example.de
102	Schmidt	schmidt@example.de

BestellNr	Datum	KundenID
5001	2024-11-01	101
5002	2024-11-02	102
5003	2024-11-03	101
5004	2024-11-04	999

```
CREATE TABLE kunde (kundenid INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100) UNIQUE CHECK (email LIKE '%@%'));
```

```
CREATE TABLE bestellung (bestellnr INT PRIMARY KEY,
datum DATE NOT NULL,
kundenid INT, CHECK (datum >= '2020-01-01'),
FOREIGN KEY (kundenid) REFERENCES kunde(kundenid));
```

Integritätsbedingungen ON UPDATE/DELETE

```
CREATE TABLE kunde (kundenid INT PRIMARY KEY,
name VARCHAR(100));



CREATE TABLE bestellung_restrict (bestellnr INT PRIMARY KEY,
kundenid INT, FOREIGN KEY (kundenid)
REFERENCES kunde(kundenid) ON DELETE RESTRICT);



CREATE TABLE bestellung_cascade (bestellnr INT PRIMARY KEY,
kundenid INT, FOREIGN KEY (kundenid)
REFERENCES kunde(kundenid) ON DELETE CASCADE);



CREATE TABLE bestellung_null (bestellnr INT PRIMARY KEY,
kundenid INT, FOREIGN KEY (kundenid) REFERENCES kunde(kundenid)
ON DELETE SET NULL);
```

Wenn ein Kunde noch Bestellungen hat, kann er nicht gelöscht werden.

Wenn ein Kunde gelöscht wird, werden alle zugehörigen Bestellungen automatisch gelöscht.

Wenn ein Kunde gelöscht wird, wird in den zugehörigen Bestellungen kundenid = NULL gesetzt.

Löschen eines Tupels aus der Vaterrelation

RESTRICTED	Ein Tupel auf welches ein Fremdschlüssel verweist, darf nicht gelöscht werden Beispiel: Kunde / Rechnung
CASCADE	Wird das Tupel gelöscht, werden auch die Referenzen gelöscht Beispiel: Exmatrikulation / Vorlesung
SET NULL **SET DEFAULT**	Wird das Tupel gelöscht, werden auch die darauf verweisenden Fremdschlüssel auf null oder einen vorher festgelegten Wert gesetzt. Beispiel: Raumbelegung / Mitarbeitende

Updaten eines Tupels aus der Vaterrelation

(PK-Änderungen in Produktivumgebungen selten)

Bsp: Wenn man den Primärschlüssel in der Vatertabelle (z. B. kunde.kundenid) ändert, kann man steuern, was mit abhängigen Fremdschlüsselwerten in der Kindtabelle passiert.

Option	Verhalten bei Update des PK	Wirkung
RESTRICT	Änderung wird verhindert, wenn es Referenzen gibt	Kunde hat Bestellungen → KundenID darf nicht geändert werden
CASCADE	Alle referenzierenden Fremdschlüssel werden mitgeändert	Kunde 1 → 5 → alle Bestellungen bekommen auch kundenid = 5
SET NULL	Fremdschlüssel wird auf NULL gesetzt	Bestellung ohne zugeordneten Kunden
SET DEFAULT	Fremdschlüssel wird auf Standardwert gesetzt	falls DEFAULT definiert ist

Bsp: Kunden und Bestellungen mit ON UPDATE CASCADE

```sql
-- 1. Vater
CREATE TABLE kunde (
    kundenid INT PRIMARY KEY,
    name VARCHAR(100)
);

-- 2. Kind mit ON UPDATE CASCADE
CREATE TABLE bestellung (
    bestellnr INT PRIMARY KEY,
    kundenid INT,
    FOREIGN KEY (kundenid) REFERENCES kunde(kundenid)
    ON UPDATE CASCADE
);

-- Väter erzeugen
INSERT INTO kunde VALUES (1, 'Vater 1');
INSERT INTO kunde VALUES (2, 'Vater 2');

-- Kinder erzeugen
INSERT INTO bestellung VALUES (1, 1);  -- verweist auf Vater 1
INSERT INTO bestellung VALUES (2, 2);  -- verweist auf Vater 2

-- Vater ändern --> Kind wird auch geändert (Referenzielle Integrität bleibt erhalten)
UPDATE kunde SET kundenid = 5 WHERE kundenid = 1;
```

Löschen eines Tupels aus der Vaterrelation – weitere Beispiele

A) **Abteilung**

Fremdschlüssel

Name	Kürzel	Leiter	Themengebiet

SET DEFAULT

Sinnvoll, wenn ein Default-Themengebiet wie „Allgemein“ gesetzt ist

B) **Prüfungs-leistungen**

Fremdschlüssel

Prüfung	Thema	Mat-Nr.	Note

CASCADE

Löscht alle Prüfungsleistungen bei Exmatrikulation

C) **Rechnungen**

Fremdschlüssel

RNr	Text	Kunden-Nr.	Betrag

RESTRICTED

Rechnung darf nicht ohne Kunde existieren

DHBW Stuttgart

Integritätsbedingungen und Mehrbenutzerbetrieb

Integritätsbedingungen legen fest, wann Zulässigkeit gegeben ist

Transitional (während Transaktionen) → A) Datenbankzustände → statisch — z.B. PK eindeutig, Betrag > 0

Transitional (während Transaktionen) → B) Zustandsübergänge → dynamisch — z.B. Trigger, die beim Ändern greifen

Temporal (zeitabhängig) → C) langfristige Datenbankentwicklungen → zeitbezogen — Regeln, die über die Zeit hinweg gelten – nicht nur im Moment der Änderung, z.B. keine doppelten Raumreservierungen

Integritätsbedingungen in SQL

Inhärente Integritätsbedingungen

Typintegrität (Festlegung von Wertebereichen von Attributen)

Schlüsselintegrität

Referentielle Integrität

Spezielle DDL-Befehle in SQL 92

NOT NULL	Wert darf nicht NULL sein
CHECK	Wert muss definierter Bedingung genügen
PRIMARY KEY	Eindeutigkeit + NOT NULL
FOREIGN KEY <...> REFERENCES ... <...>	FK verweist auf gültigen PK in anderer Tabelle
ASSERTION	Globale Bedingungen über mehrere Tabellen (selten unterstützt)

Assertions in SQL

Wird in MySQL, SQLite, PostgreSQL nicht unterstützt → „Simulation“ mit Trigger

ASSERTION
in IBM DB2:

```
1 CREATE ASSERTION Preisbegrenzung
2 CHECK (
3   (SELECT SUM(preis) FROM buch) <= 10000
4 );
5
```

beliebige boole‘sche Bedingung

Behandlung von Verletzungen **der Integrität**

Auslösendes Ereignis → **on update / on delete** → Konsistenzerhaltende Handlung

Cascade / Set NULL / set default / no action

Transaktionen: deferred / immediate

Wird am Ende der Transaktion geprüft

Muss sofort gelten

Assertions in SQL – Beispiel

```sql
CREATE TABLE PRUEFEN (
    MatNr   INTEGER,
    VorlNr  INTEGER,
    PersNr  INTEGER,
    Note    NUMERIC(2,1) CHECK (Note BETWEEN 1.0 AND 4.0),
    PRIMARY KEY (MatNr, VorlNr),
    FOREIGN KEY (MatNr)  REFERENCES Studenten  ON DELETE CASCADE,
    FOREIGN KEY (VorlNr) REFERENCES Vorlesungen,
    FOREIGN KEY (PersNr) REFERENCES Professoren ON DELETE SET NULL
);
```

Assertion ←→Trigger

Trigger reagieren auf etwas (→ Ereignis)
Assertions gelten immer und global (→ Zustand).

Merkmal	Trigger	Assertion
Auslöser:	Reagiert auf Änderungen (INSERT/UPDATE/DELETE)	Wird automatisch bei jeder DB-Änderung
Gültigkeit	Gilt für eine bestimmte Tabelle	Gilt datenbankweit / global
Unterstützung:	in allen gängigen DBMS verfügbar	Wenige, z.B. IDB DB2

Integritätsbedingungen – Klassifikation

1. Studenten werden durch Name, Vorname und Geburtsdatum identifiziert.
2. Der Kontostand eines Girokontos darf nicht unter die Kreditlinie von 5000 EURO fallen.

betrifft alle Tupel (-paare)

3. Der Kontostand von Herrn X darf nicht unter 0 fallen.

betrifft *ein* Tupel

4. Der Minimalpreis eines antialkoholischen Getränks muss unter dem Minimalpreis eines alkoholischen Getränks liegen.

betrifft eine Relation / Paare von Teilmengen von Tupeln

5. Es dürfen nur Waren bestellt werden, für die es mindestens einen Lieferanten gibt.

Relationenübergreifend

6. Der Mietzins darf dieses Jahr nicht erhöht werden.

betrifft einen Zustandsübergang

7. Kunden dürfen nur gelöscht werden, wenn sie keine Waren mehr bestellt haben.

direkt der Operation zugeordnet

8. Kunden müssen gelöscht werden, wenn sie keine Waren mehr bestellt haben.

langfristig zu überprüfen !

9. Der Mietzins darf innerhalb von 3 Jahren um nicht mehr als 10 % erhöht werden.

Integritätsregel (Handlung)

Integritätsbedingungen – in SQL

1. Studenten werden durch Name, Vorname und Geburtsdatum identifiziert. → **Durch Festlegung eines Schlüssels**
2. Der Kontostand eines Girokontos darf nicht unter die Kreditlinie von 5000 EURO fallen. →
   Kontostand numeric(20,2)
   CHECK (Kontostand > -5000)
3. Der Kontostand von Herrn X darf nicht unter 0 fallen. →
   Kontostand numeric(20,2)
   CHECK (NOT (Name = „X“) OR
   Kontostand > 0)
4. Der Minimalpreis eines antialkoholischen Getränks muss unter dem Minimalpreis eines alkoholischen Getränks liegen. →
   CREATE ASSERTION PubLaw CHECK
   ((SELECT min(Preis) FROM Getränk WHERE alk > 0.0)
   > (SELECT min(Preis) FROM Getränk WHERE alk = 0.0))
5. Es dürfen nur Waren bestellt werden, für die es mindestens einen Lieferanten gibt.
6. Der Mietzins darf dieses Jahr nicht erhöht werden. → **Dynamisch !?** **Trigger Konzept**
7. Kunden dürfen nur gelöscht werden, wenn sie keine Waren mehr bestellt haben.
8. Kunden müssen gelöscht werden, wenn sie keine Waren mehr bestellt haben.
9. Der Mietzins darf innerhalb von 3 Jahren um nicht mehr als 10 % erhöht werden. → **Nicht unterstützbar!**

Trigger

Wird von vielen DBS unterstützt, ist jedoch erst in SQL3 (SQL 99) standardisiert.

⇨ Bei **Tabellenänderungen** wird eine Aktivität ausgelöst

Aufruf von „stored procedures“ → Beliebig umfangreiche Aktionen!

➡ **Beispiel**

```
1 CREATE TRIGGER blockiere_studentenloeschung
2 BEFORE DELETE ON Studenten
3 FOR EACH ROW
4 BEGIN
5     CALL check_studentenloeschung(OLD.MatrNr);
6 END;
```

BEFORE/ AFTER

UPDATE/DELETE/INSERT

Vielen Dank für Ihre Aufmerksamkeit!